UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT

Data Structure and Algorithm

Laboratory Activity No. 13

Tree Algorithm

Submitted by:
Aquino, Jester J.

Instructor:
Engr. Maria Rizette H. Sayo

November 9, 2025

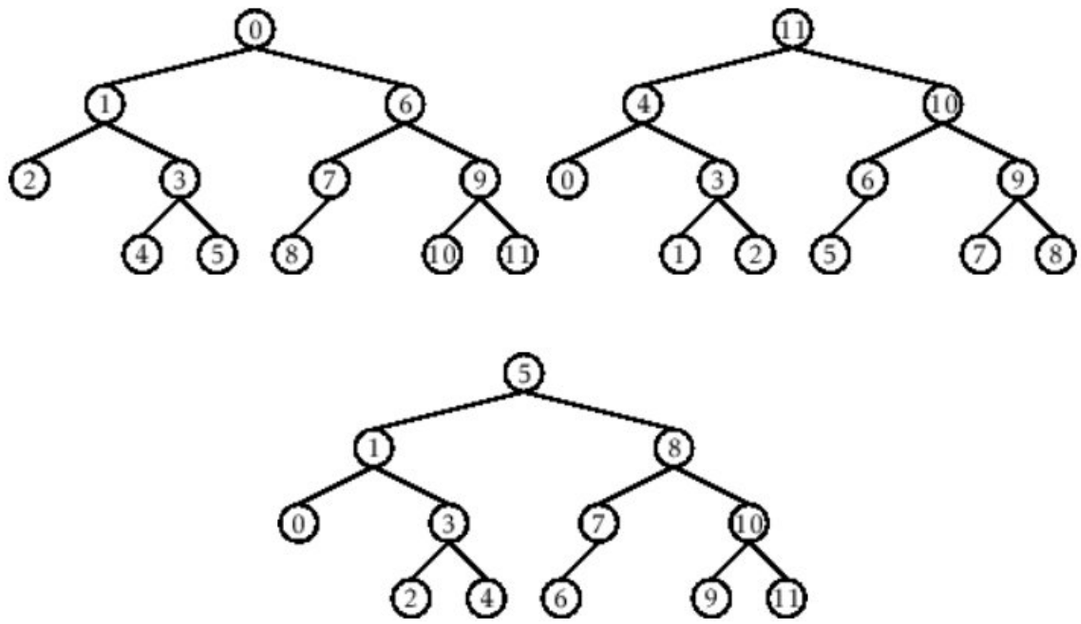
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def remove_child(self, child_node):
        self.children = [child for child in self.children if child != child_node]
```

```
    def traverse(self):
        nodes = [self]
        while nodes:
            current_node = nodes.pop()
            print(current_node.value)
            nodes.extend(current_node.children)

    def __str__(self, level=0):
        ret = "  " * level + str(self.value) + "\n"
        for child in self.children:
            ret += child.__str__(level + 1)
        return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()
```

Questions:

1. When would you prefer DFS over BFS and vice versa?
2. What is the space complexity difference between DFS and BFS?
3. How does the traversal order differ between DFS and BFS?
4. When does DFS recursive fail compared to DFS iterative?

III. Results

1. When would you prefer DFS over BFS and vice versa?

Answer: DFS (Depth-First Search) Memory is limited less space on wide trees and perform task like topological sorting, cycle detection, or checking connectivity and also it explore as deep as possible before backtracking. While BFS (Breadth-First Search) The tree graph is shallow few levels or many nodes per level and it exploring all neighbors first before going deeper.

2. What is the space complexity difference between DFS and BFS?

Answer: DFS where h is tree height (for recursive stack or explicit stack) because the nodes on the current path in memory while the BFS where w is the maximum width of the tree (number of nodes at the widest level) because it stores all nodes at the current level in queue

3. How does the traversal order differ between DFS and BFS?

Answer: DFS order from Root - Child 1 - Grandchild 1 - Child 2 - Grandchild 2 it goes deeper first exploring before siblings while BFS order from Root - Child 1 - Child 2 - Grandchild 1 - Grandchild 2 it goes wide first exploring siblings before children.

4. When does DFS recursive fail compared to DFS iterative?

Answer: Recursive DFS can fail when the tree is very deep it causing a stack overflow and you need to explicit the control over the traversal order while the Iterative it using stack preferred when the structure can be very deep, avoid recursion depth limits and you can customize traversal behavior.

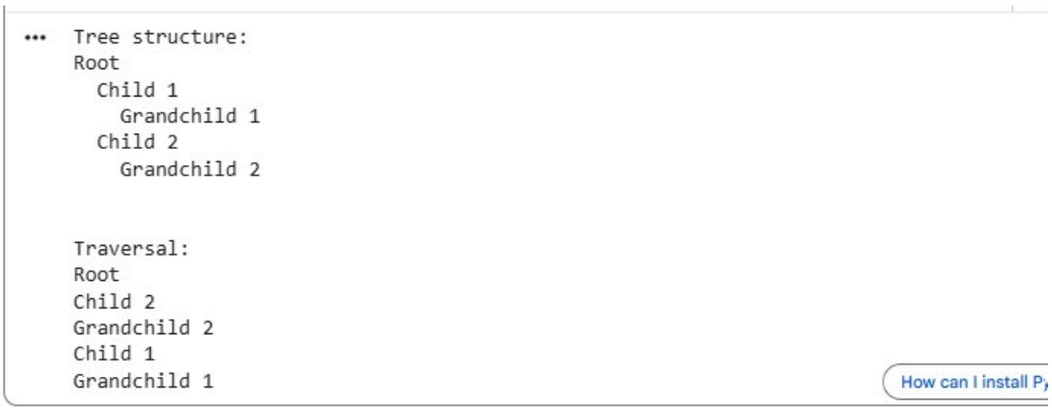


Figure 1 Screenshot of program

IV. Conclusion

In this activity, we explored how to create and traverse a tree data structure using Python. We learned how each node can have multiple children and how traversal methods like Depth-First Search (DFS) and Breadth-First Search (BFS) work differently. Overall this activity helped in understanding how tree traversal works.

References

[1] GeeksforGeeks. (2023, June 15). *Difference between BFS and DFS*.
https://www.geeksforgeeks.org/difference-between-bfs-and-dfs/